

# INFORME EJECUTIVO Y TÉCNICO DE PENTESTING



Evaluación de Seguridad Ofensiva, Hackeo Ético  
y Verificación OWASP ASVS v4.0.3





Índice / Tabla de Contenidos

|                                                          |         |
|----------------------------------------------------------|---------|
| 1. Resumen Ejecutivo y Ficha Técnica                     | Pág. 3  |
| 2. Metodología de Evaluación y Estándares Aplicados      | Pág. 4  |
| 2.1 Marcos Normativos Utilizados                         | Pág. 4  |
| 2.2 Escala de Severidad y Prioridad                      | Pág. 4  |
| 3. Métricas y Distribución Estadísticas de Hallazgos     | Pág. 5  |
| Gráficos de Distribución (Barras — Estado del Ticket)    | Pág. 6  |
| Gráficos de Distribución (Torta — Severidad)             | Pág. 7  |
| 4. Alcance Técnico y Componentes Evaluados               | Pág. 8  |
| 5. Detalle de Vulnerabilidades e Incidencias Registradas | Pág. 9  |
| Prueba de Concepto (PoC) paso a paso                     | Pág. 10 |
| 6. Conclusiones y Plan de Remediación Recomendado        | Pág. 11 |

# 1. Resumen Ejecutivo y Ficha Técnica

El presente informe consolida los resultados y hallazgos identificados durante la ejecución de las pruebas de penetración (Pentest) y evaluación de seguridad sobre la plataforma evaluada.

El objetivo principal de esta evaluación es identificar de forma proactiva vulnerabilidades de seguridad que pudieran ser explotadas por actores maliciosos, permitiendo a la organización implementar los controles correctivos pertinentes antes de su despliegue o puesta en producción.

| Parámetro / Campo           | Detalle de la Evaluación                          |
|-----------------------------|---------------------------------------------------|
| Tipo de Evaluación          | Penetration Testing (Box Gris / Caja Negra)       |
| Estándar Principal          | OWASP ASVS 4.0.3 (Level 2/3) & OWASP Top 10       |
| Período de Pruebas          | Octubre 2026 — Fechas programadas con el cliente  |
| Entorno de Evaluación       | Ambiente de Staging / Pre-Producción / Producción |
| Clasificación del Documento | <b>ESTRICTAMENTE CONFIDENCIAL</b>                 |

## VALOR PARA QUIEN LO LEE

Esta página le permite a un directivo decidir si el proyecto puede pasar a producción sin leer una sola línea técnica: ve el alcance, los estándares usados y quién firma el documento. Así un hallazgo crítico nunca se pierde entre jerga técnica.

## 2. Metodología de Evaluación y Estándares Aplicados

Las pruebas se ejecutaron combinando análisis estático (SAST), análisis dinámico (DAST) y pruebas de penetración manuales profundas, para reducir falsos positivos y garantizar la exhaustividad del análisis.

### 2.1 Marcos Normativos Utilizados

#### OWASP ASVS v4.0.3

Verificación estructurada de controles de seguridad en arquitectura, autenticación, control de acceso y criptografía.

#### OWASP Top 10

Clasificación de las vulnerabilidades web más críticas: inyecciones, fallos de control de acceso, criptografía deficiente, entre otras.

#### PTES — Penetration Testing Execution Standard

Guía de fases del pentest: reconocimiento, descubrimiento, explotación y post-explotación.

### 2.2 Escala de Severidad y Prioridad (CVSS v3.1)

Los hallazgos se evalúan bajo la matriz CVSS v3.1, clasificándose en cuatro niveles:

#### CRÍTICA · 9.0–10.0

Ejecución remota de código o acceso total al sistema sin autenticación.

#### ALTA · 7.0–8.9

Compromiso grave de confidencialidad o integridad con impacto sustancial.

#### MEDIA · 4.0–6.9

Fallos de lógica o bypass de controles que requieren condiciones previas.

#### BAJA · 0.1–3.9

Revelación menor de información técnica o configuraciones no óptimas.

#### VALOR PARA QUIEN LO LEE

Esta escala es la misma lógica con la que priorizás bugs en QA: un hallazgo Crítico bloquea el release igual que un bug Bloqueante. Clasificar con CVSS le da mejor claridad a algo que, en el reporte de bugs, muchas veces depende del criterio del tester.

### 3. Métricas y Distribución Estadística de Hallazgos

A continuación se consolidan las métricas del estado actual de los hallazgos registrados durante la auditoría de ejemplo (laboratorio PortSwigger — Control de Acceso / IDOR).



#### Distribución por estado del ticket

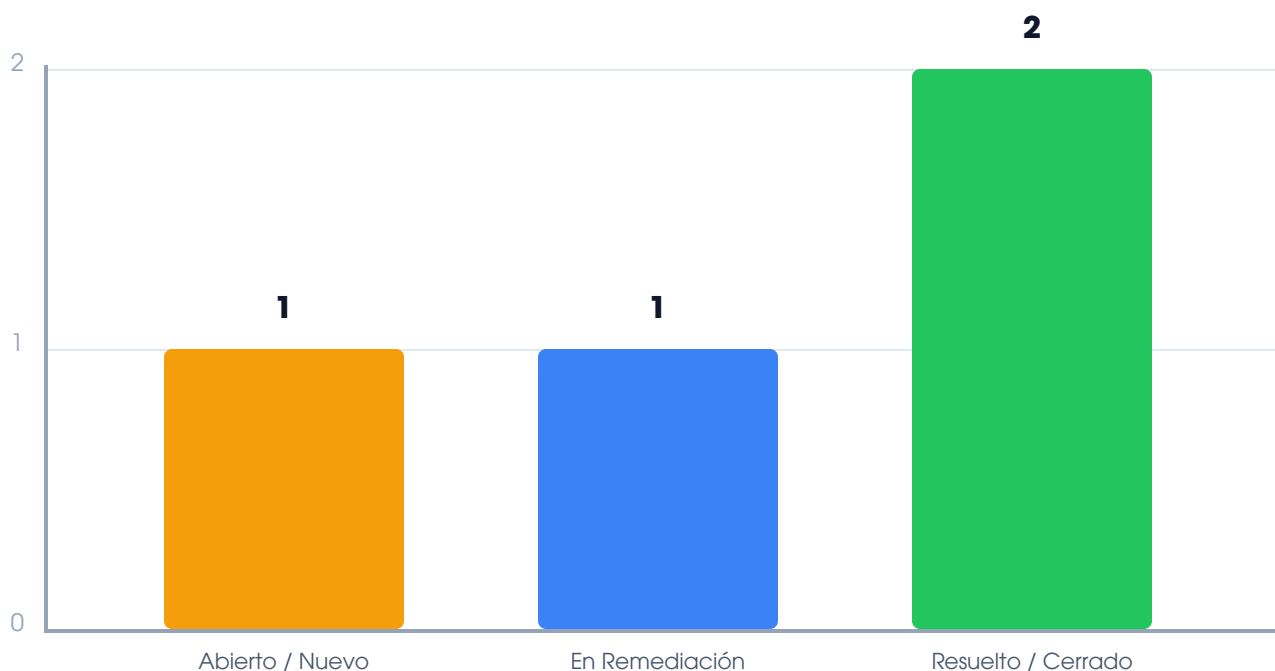
| Estado del hallazgo | Cantidad | Porcentaje |
|---------------------|----------|------------|
| Abierto / Nuevo     | 1        | 25%        |
| En Remediación      | 1        | 25%        |
| Resuelto / Cerrado  | 2        | 50%        |
| Total               | 4        | 100%       |

#### VALOR PARA QUIEN LO LEE

Las métricas le dan al líder una foto rápida del estado de seguridad sin leer cada ticket: cuántos hallazgos hay, qué tan graves son y cuántos siguen abiertos. Es el equivalente al dashboard de bugs que un QA Lead revisa antes de aprobar un release.

## Distribución por Estado del Ticket

El gráfico de barras muestra cómo se reparten los hallazgos según su estado de gestión al cierre de la auditoría.



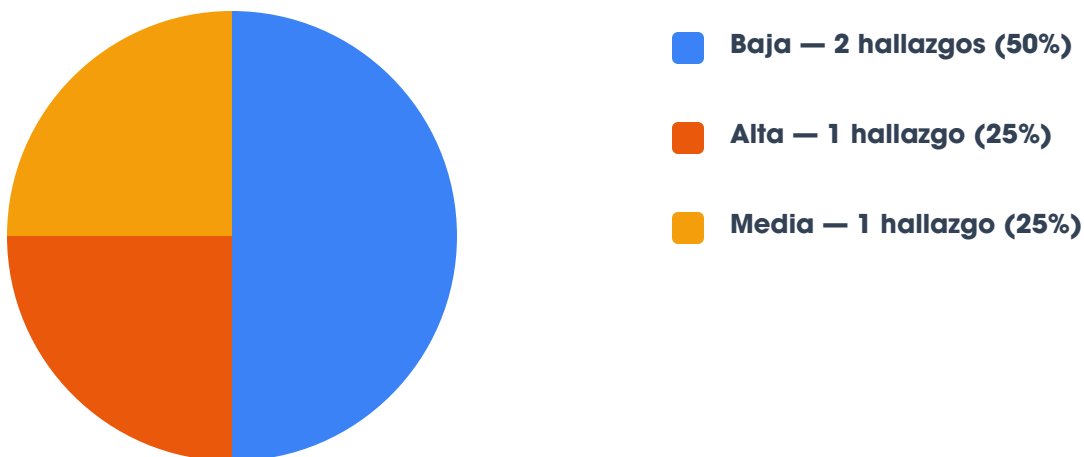
Cada barra representa la cantidad de hallazgos en ese estado. La mitad (50%) ya fue resuelta durante la ventana de pruebas; el resto quedó en seguimiento.

### VALOR PARA QUIEN LO LEE

Un gráfico de estado es el equivalente al vistazo general de bugs: si el equipo está cerrando tickets o si se acumulan. Ideal para mostrar en un daily o en una reunión de seguimiento sin abrir ticket por ticket.

## Distribución por Severidad

La torta muestra la proporción de hallazgos según su nivel de severidad CVSS.



El 50% de los hallazgos son de severidad Baja; el único hallazgo Alto (el IDOR) concentra el mayor riesgo y se detalla en la sección 5.

### VALOR PARA QUIEN LO LEE

La torta de severidad es como el reparto de bugs por prioridad: le comunica al negocio prioridades. Un solo bug Alto puede justificar frenar un release aunque la mayoría sean Bajos.

## 4. Alcance Técnico y Componentes Evaluados

La evaluación abarcó los siguientes vectores de ataque y componentes de software:

### **Autenticación y Gestión de Sesiones**

Políticas de contraseñas, tokens JWT/OAuth, almacenamiento de cookies y protección contra fuerza bruta.

### **Control de Acceso e Impersonación**

Elevación de privilegios vertical y horizontal (IDOR / BOLA / BFLA) — foco principal de este informe.

### **Validación de Entradas y Sanitización**

Búsqueda de inyecciones (SQLi, XSS, Command Injection, SSRF).

### **Seguridad en Comunicaciones y Criptografía**

Cifrado de datos en tránsito y en reposo, y algoritmos utilizados.

### **Manejo de Errores y Registro de Eventos**

Exposición de trazas de pila (stack traces) y logs de auditoría.

### **VALOR PARA QUIEN LO LEE**

Definir el alcance es como escribir la sección «Scope / Out of Scope» de un plan de pruebas: evita reclamos por algo que nunca entró en la evaluación y deja claro, qué quedó cubierto y qué no.

## 5. Detalle de Vulnerabilidades e Incidencias Registradas

Se detalla el hallazgo principal identificado, con su descripción técnica, clasificación normativa y desglose de severidad CVSS.

VULN-001 · IDOR — Exposición de transcripciones de chat

ALTA · CVSS 7.1

|                 |                                                            |
|-----------------|------------------------------------------------------------|
| CWE             | CWE-639 — Authorization Bypass Through User-Controlled Key |
| OWASP Top 10    | A01:2021 — Broken Access Control                           |
| Vector CVSS 3.1 | AV:N / AC:L / PR:L / UI:N / S:U / C:H / I:L / A:N          |
| Estado          | En Remediación                                             |

### Descripción técnica

La función de chat en vivo almacena cada transcripción en el servidor y la entrega mediante una URL con un identificador numérico secuencial (/download-transcript/2.txt). La aplicación no verifica que la transcripción solicitada pertenezca al usuario autenticado: basta con decrementar el número del archivo para acceder a las conversaciones de otros usuarios.

### Impacto

La transcripción del usuario «carlos» contiene su contraseña en texto plano. Un atacante autenticado con una cuenta de bajo privilegio puede leerla y tomar control total de esa cuenta (account takeover), accediendo a los datos personales y funciones de la víctima.

#### VALOR PARA QUIEN LO LEE

Este ticket tiene todo lo que exijo en un bug report serio: ID único, severidad objetiva, clasificación e impacto de negocio. Solo que acá el «resultado esperado» es que el sistema NIEGUE el acceso a un recurso ajeno.

## Prueba de Concepto (PoC) — Paso a Paso

Reproducción completa del hallazgo en el entorno de laboratorio. Cada paso es reproducible y verificable.

- 1 Iniciar sesión en la aplicación con las credenciales de bajo privilegio provistas (wiener:peter).
- 2 Abrir «Live chat», enviar un mensaje y hacer clic en «View transcript».
- 3 Observar que la transcripción se descarga desde /download-transcript/2.txt (identificador numérico secuencial).
- 4 Modificar el identificador del archivo de 2.txt a 1.txt y volver a solicitar la URL.
- 5 El servidor responde con la transcripción de «carlos», que contiene su contraseña en texto plano.
- 6 **Iniciar sesión como «carlos» con esa contraseña → acceso total a la cuenta ajena. Laboratorio resuelto.**

```
GET /download-transcript/1.txt HTTP/1.1
Host: TU-LAB-ID.web-security-academy.net
Cookie: session=<sesión de wiener>

--- Respuesta del servidor ---
HTTP/1.1 200 OK

CONNECTED: -- Now chatting with Hal Pline --
You: Hola
carlos: Mi contraseña es: *****
```

### VALOR PARA QUIEN LO LEE

Una PoC paso a paso: pasos numerados, dato de entrada y resultado observado. Para que el equipo de dev pueda reproducir el hallazgo

## 6. Conclusiones y Plan de Remediación Recomendado

Se recomienda priorizar la corrección del hallazgo Alto antes del pase a producción. A continuación, la corrección técnica y el plan de verificación.

### Corrección recomendada — Validar la propiedad del recurso en el backend

```
// middleware/validateOwnership.js
function validateOwnership(req, res, next) {
  const requestedId = req.params.transcriptId;
  const owned = req.user.transcriptIds;
  if (!owned.includes(requestedId))
    return res.status(403).json({ error: "Acceso denegado" });
  next();
}
// Ruta protegida: authenticateSession + validateOwnership
// se ejecutan antes de downloadTranscript
```

### Plan de acción y timeline

|                             |                                                                                |
|-----------------------------|--------------------------------------------------------------------------------|
| <b>INMEDIATO (0-24 h)</b>   | Deshabilitar el endpoint vulnerable o aplicar un control de acceso temporal.   |
| <b>CORTO PLAZO (1-3 d)</b>  | Implementar la validación de propiedad del recurso en el backend (ver código). |
| <b>VERIFICACIÓN (3-5 d)</b> | Ejecutar el re-test del hallazgo y cerrar formalmente el ticket.               |

### Criterios de verificación (re-test)

- Un usuario no puede acceder a transcripciones que no le pertenecen (respuesta 403).
- El identificador de recurso se valida contra el dueño y/o deja de ser secuencial y adivinable.
- Ninguna contraseña viaja ni se almacena en texto plano dentro de las transcripciones.

#### Elaborado por

María de los Ángeles Acuña  
QA & Analista de Ciberseguridad (Trainee)

#### Revisado y aprobado por

Lead Security Auditor / CISO  
Firma: \_\_\_\_\_

#### VALOR PARA QUIEN LO LEE

El valor de esta sección es que convierte un hallazgo en un ticket accionable y medible: el dev sabe qué tocar, el líder sabe cuándo, y el re-test define objetivamente cuándo el bug está realmente cerrado.